TENDERHUB ENGINEERING PUBLICATIONS • TECHNICAL REPORT SERIES

TenderHub Developer Handover Manual & Engineering Guide

Architecture, Operations, Automated Crawling, Billing, and AI Intelligence

Author: Lead Systems Architect **Date:** October 2026 **Classification:** Internal Technical Manual **Version:** 2.4.0

ABSTRACT

This document serves as the comprehensive engineering handover specification for the incoming technical team. It details the operational setup, monorepo architecture, crawler automation pipeline, payment reconciliation guarantees, cloud infrastructure operating costs, and the technical roadmap for finalizing the contractor AI analysis subsystem.

1. Executive Summary & Monorepo Layout

TenderHub is an enterprise-grade automated procurement intelligence platform designed to eliminate the operational friction of bidding on Indian government and public works procurement notices (with initial high-fidelity ingestion tailored for the ASP.NET NIC-GEP infrastructure, specifically `jktenders.gov.in`).

The software architecture is decoupled into three operational tiers:

- **`backend/`**: Node.js (v20+) running Express 5, Mongoose 9, Redis with BullMQ, Playwright Chromium for portal scraping, Razorpay SDK with webhook verifiers, AiSensy WhatsApp engine, and Google Gemini / OpenAI contractor intelligence services.
- **`frontend/`**: Single-page React 19 application built with Vite 8, React Router v7, Zustand 5, TanStack Query 5, and Tailwind CSS.
- **`admin-panel/`**: Isolated administrative console running on Vite for manual scrape triggers, system telemetry, and database oversight.

```
tenderHub/
├── backend/           # Express 5 REST API, BullMQ crawlers, AI intelligence
├── frontend/          # React 19 + Vite 8 SPA (Client Workspace)
├── admin-panel/       # Isolated administrative dashboard (Port 5174)
├── docs/              # Technical handover manuals, PDF reports, and architecture guides
└── README.md          # Engineering repository overview
```

2. Local Development Setup & All Execution Commands

The following tables specify all execution commands required to run, build, test, and maintain the platform.

2.1 Backend Commands (`cd backend`)

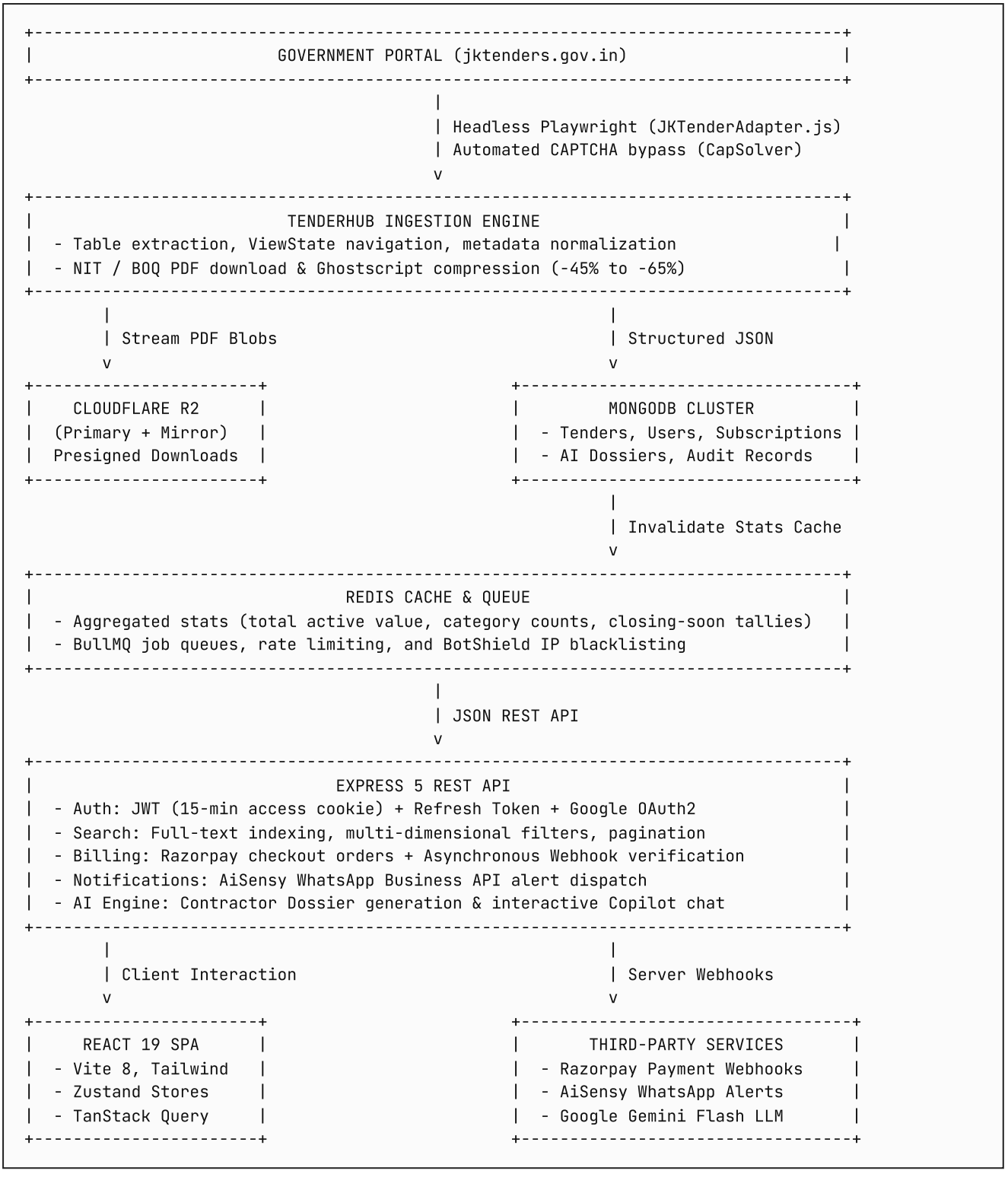
Command	Description & Purpose	Runtime Context
`npm run dev`	Starts API server with Nodemon live-reload on port 8000	Local Development
`npm start`	Executes `node src/server.js` in production mode	Production Container
`npm run worker`	Starts standalone BullMQ background worker queue processor	Background Workers
`npm run scrape:latest`	Fetches recently published notices from the portal (last 24-48 hrs)	Daily Cron (09:00, 14:00)
`npm run scrape:all`	Crawls active tenders across all government departments	Initial Seeding & Audit
`npm run scrape:date`	Interactive prompt to crawl notices published on a specific historical date	Manual Reconciliation
`npm run fetch:pending-docs`	Re-attempts downloading NIT and BOQ documents for incomplete records	Nightly Sweep (01:00)
`npm run backup:db`	Exports MongoDB collections to local JSON/BSON archives	Disaster Recovery
`npm run restore:db`	Restores MongoDB collections from local backup archive	Database Recovery
`npm run sync:mirror`	Synchronizes primary Cloudflare R2 bucket to secondary backup R2 bucket	Off-site Redundancy
`npm run purge:expired`	Marks notices past bid closing date as `status: 'EXPIRED'`	Daily Maintenance (03:00)
`npm run reconcile:dates`	Audits and normalizes non-standard portal timestamps across records	Data Hygiene
`npm run manuals:all`	Compiles all architectural PDF manuals directly into `docs/`	Documentation Build

2.2 Frontend & Admin Commands

Directory	Command	Description
`frontend/`	`npm run dev`	Runs Vite development server on `http://localhost:5173`
`frontend/`	`npm run build`	Compiles production bundle to `frontend/dist/`
`frontend/`	`npm run preview`	Previews compiled production build locally on port 4173
`frontend/`	`npm run lint`	Runs ESLint verification across all JSX/JS sources
`admin-panel/`	`npm run dev`	Runs Admin console on `http://localhost:5174`

3. System Architecture & End-to-End Data Flow

The platform operates on a coordinated pipeline connecting automated web crawling, cloud object storage, distributed caching, and client discovery:



3.1 Payment Verification & Razorpay Webhook Guarantee

Payment processing utilizes dual-path validation to prevent dropped subscriptions when users experience network disconnections:

1. **Synchronous Client Callback**: The frontend submits `orderId`, `paymentId`, and `signature` to `POST /api/v1/billing/verify-payment`. The backend validates the HMAC-SHA256 signature using `RAZORPAY_KEY_SECRET`.
2. **Asynchronous Server-to-Server Webhook**: Razorpay sends an HTTP POST event (`payment.captured` or `order.paid`) to `POST /api/v1/billing/webhook`. The backend verifies `X-Razorpay-Signature` using `RAZORPAY_WEBHOOK_SECRET`. If the client dropped connection before completing the redirect, the webhook automatically activates the contractor's plan and credits their AI tokens.

4. Contractor AI Intelligence Subsystem: Status & Continuation Plan

The AI subsystem is located in `backend/src/services/contractorAgent.service.js`, `backend/src/models/TenderAiDossier.js`, and `frontend/src/pages/Services.jsx`.

4.1 Current Implementation Status

- **Completed Schemas**: `TenderAiDossier` (pre-bid checklist, BOQ materials, milestones, risk flags) and `ContractorAgentProfile` (turnover, solvency, equipment, credit quota).
- **Multi-Tier Execution**: Tier 1 (Google Gemini Flash) → Tier 2 (OpenAI gpt-4o-mini) → Tier 3 (Deterministic Rule-Based Civil Engineering Engine).
- **Zero-Failure Guarantee**: If external API keys are missing or quota is exhausted, Tier 3 deterministically calculates realistic material schedules and milestones, ensuring the UI remains 100% operational.
- **Contractor Gap Analysis**: Compares tender value against contractor profile (Turnover: 120%, Solvency: 40%, Single Work: 50%, Equipment registry) and flags qualification deficiencies.
- **Copilot Chat**: Interactive conversation session per tender via `POST /api/v1/services/chat/:tenderId`.

4.2 What Is Incomplete & What the Incoming Developer Must Do

> **NOTICE TO INCOMING DEVELOPER**
>
> The AI subsystem currently relies on the deterministic engine because no live Google Gemini API key is configured in production, and full NIT PDF documents stored in Cloudflare R2 are not yet piped into the prompt.

Follow these steps to complete the feature:

1. **Configure Live Gemini API Key**: Obtain a key from Google AI Studio and configure `GEMINI_API_KEY` and `GEMINI_MODEL=gemini-1.5-flash` in `backend/.env` and Render Dashboard.
2. **Implement Cloudflare R2 PDF Text Extraction**: In `contractorAgent.service.js`, stream the tender's NIT PDF from Cloudflare R2 using `GetObjectCommand`, extract text with `pdf-parse`, and pass the text directly into the Gemini prompt. Gemini 1.5 Flash supports up to 1,000,000 tokens, effortlessly ingesting 100-page tender documents.
3. **Enable Gemini Context Caching**: Cache the ingested tender document using Google's Context Caching API. Repeat queries from other contractors incur 75% lower token fees and lower latency.
4. **Add Frontend Custom Rate Overrides**: In `frontend/src/pages/Services.jsx`, permit contractors to edit material rates in the BOQ table to recalculate their expected bidding margins and export to Excel (.xlsx).

5. Server Hosting & Cloud Infrastructure Operating Costs

The table below provides a full financial projection for running TenderHub in production across all cloud vendors:

Infrastructure Component	Provider & Specification	Monthly Cost (USD)	Monthly Cost (INR)
Backend Web Service	Render Starter / Standard Instance	$7.00 - $25.00	₹585 - ₹2,090
Frontend Static CDN	Render / Vercel Hobby Tier	$0.00 (Free)	₹0
Application Database	MongoDB Atlas (M0 Free / M2 Shared)	$0.00 - $9.00	₹0 - ₹750
Caching & Queues	Upstash Redis Serverless	$0.00 - $5.00	₹0 - ₹420
Document Object Storage	Cloudflare R2 (10 GB Free, Zero Egress)	$0.00 - $1.00	₹0 - ₹85
WhatsApp Alert Dispatch	AiSensy WhatsApp Business API	~$15.00 - $25.00	₹1,250 - ₹2,100
Transactional Email	Brevo (300 emails/day free)	$0.00 (Free)	₹0
Crawler CAPTCHA Solvers	CapSolver API ($0.80 / 1k solves)	~$2.00 - $4.00	₹165 - ₹335
AI LLM Inference	Google Gemini 1.5 Flash Token Billing	~$5.00 - $12.00	₹420 - ₹1,000
Disaster Recovery Backup	Telegram Bot API Snapshot Offsite	$0.00 (Free)	₹0
Payment Processing	Razorpay Standard Commission	2% of GMV	2% of GMV
TOTAL ESTIMATED MONTHLY	**All Platform Services**	**~$29.00 - $81.00**	**₹2,420 - ₹6,780**

Unit Economics: With TenderHub Pro subscriptions priced at ₹999/month and ₹2,999/year, the platform operates at an exceptionally healthy gross contribution margin exceeding 90%.

6. Production Deployment & Security Runbook

6.1 Production Environment Variables

```
NODE_ENV=production
PORT=5000
MONGO_URI=mongodb+srv://...
SECONDARY_MONGO_URI=mongodb+srv://... (Failover DB)
REDIS_URL=rediss://...
JWT_ACCESS_SECRET=[REDACTED]
JWT_REFRESH_SECRET=[REDACTED]
BACKEND_URL=https://tenderhub-4nbc.onrender.com
FRONTEND_URL=https://tenderhub.in
CORS_ORIGIN=https://tenderhub.in
RAZORPAY_KEY_ID=[REDACTED]
RAZORPAY_KEY_SECRET=[REDACTED]
RAZORPAY_WEBHOOK_SECRET=[REDACTED]
AISENSY_API_KEY=[REDACTED]
CAPSOLVER_API_KEY=[REDACTED]
GEMINI_API_KEY=[REDACTED]
GEMINI_MODEL=gemini-1.5-flash
R2_ACCOUNT_ID=2b0771a6f411ca5ab5e1c9bfd21b97ba
R2_ACCESS_KEY_ID=[REDACTED]
R2_SECRET_ACCESS_KEY=[REDACTED]
R2_BUCKET_NAME=tenderhub
SCRAPER_HEADLESS=true
```

6.2 BotShield & Security Protections

The security layer in `backend/src/middleware/botShield.middleware.js` actively monitors incoming traffic:

- **User-Agent Whitelisting**: Rejects unauthorized automated scraping frameworks (Scrapy, requests, generic Python bots).
- **Honeypot Traps**: Requests attempting to access vulnerability probe URLs (`/.env`, `/wp-login.php`) trigger immediate and permanent IP blacklisting stored in Redis.
- **Rate Limiting**: Limits requests per IP address to safeguard database aggregation queries.